

11d — Serverless

Key Concepts

- **Serverless** — deploy code, cloud provider manages all infrastructure, scaling, and execution
- **FaaS (Functions as a Service)** — run small, single-purpose functions triggered by events
- **Cold start** — latency penalty (200ms–2s) when function spins up after being idle
- **Stateless** — no memory between executions; state must live in external storage
- Name is misleading — servers exist, you just don't manage them

How It Works

Traditional server:

Provision server → runs 24/7 → pay 24/7 (even when idle)

Serverless:

Event happens → cloud spins up function → runs → shuts down
No traffic → nothing running → pay nothing

Event Triggers

Trigger	Example
HTTP request	API call hits your function
Message queue	New Service Bus message → function fires
File upload	Image uploaded → resize function fires
Scheduled timer	Midnight → cleanup function fires
Database change	New Cosmos DB record → sync function fires

Providers

Provider	Product
AWS	Lambda (15 min timeout)
Azure	Azure Functions (10 min timeout)
Google Cloud	Cloud Functions
Cloudflare	Workers

Strengths vs Weaknesses

Strength	Weakness
No infrastructure management	Cold start latency (200ms–2s)
Auto-scaling — automatic	Timeout limits — not for long-running work
Pay per execution only	Stateless — state needs external storage
Fast to build	Vendor lock-in
	Hard to test locally

Cold Start

What happens:

```
Function idle → container spun down
Next request → spin up container → load runtime → load code → execute
= 200ms to 2s latency penalty
```

Mitigations:

- **Provisioned concurrency** (AWS) / **Premium plan** (Azure) — keeps functions warm
- Keep functions lightweight — less code = faster startup
- Use fast-startup languages — Node.js, Python, Go start faster than Java

Serverless vs Microservices

	Microservices	Serverless
Unit	Running service (container)	A function
Runs	Continuously	Only when triggered
State	Can hold some in-memory state	Stateless — external DB only
Cost	Pay for running containers 24/7	Pay per execution
Best for	Long-running, complex services	Event-driven, sporadic, short tasks

Industry Examples

- **Netflix** — AWS Lambda triggers encoding pipeline when video is uploaded
- **Coca-Cola** — replaced 24/7 vending machine backend with Lambda; cost dropped from \$13,000/month to \$4,500/month
- **Notification services** — ideal serverless use case; only runs when a message arrives

Interview Questions

Q: What is serverless? A: A model where you deploy functions and the cloud provider manages all infrastructure, scaling, and execution. You pay only when the function runs.

Q: What is a cold start? A: When a function hasn't run recently, the cloud spins down the container. Next request pays a latency penalty of 200ms–2s while it spins back up.

Q: When would you NOT use serverless? A: Long-running processes (beyond timeout limits), latency-sensitive APIs where cold starts are unacceptable, or consistently high traffic where always-on containers are cheaper.

Q: What does stateless mean in serverless? A: Each function execution is independent — no shared memory between calls. Any state must be stored externally in a database or cache.